Bedrock

🧠Amazon Bedrock

🔷 Simple Definition:

Amazon Bedrock is a service that lets you use **pre-trained foundation models** (like ChatGPT, Claude, Mistral, or Amazon Titan) **without training your own model** or managing servers.

Think of it as:

> 💬 "Hey AWS, I want to use a smart AI model like ChatGPT in my app — give me the power, but don't make me build or host anything."

🧪 Why Would You Use Bedrock?

Use Bedrock if:

- You want to build a **chatbot**, **text summarizer**, **image generator**, **code assistant**, etc.
- You don't want to train your own model — you just want to **plug into a powerful LLM**.
- You're building a **LangChain app** and want access to high-quality commercial models from **Anthropic**, **Meta**, **Cohere**, **AI21**, etc.

You **don't** use Bedrock for:

- Traditional ML (XGBoost, Linear Regression, etc.) → Use **SageMaker**.
- Tabular data workflows → Use **Glue**, **Athena**, or **SageMaker**.

🖥️ Visual Analogy:

Imagine this:

Service	Analogy
SageMaker	You're baking your own cake. You gather ingredients (data), mix, bake (train model), and serve (deploy).
Bedrock	You're ordering a cake from a bakery. It's ready-made by experts (pretrained LLMs). You just choose the flavor (Claude, Mistral, Titan, etc.) and use it.

Llama3 Model In Amazon bedrock

Refer → https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-meta.html

```
import boto3
import json
```

```
client = boto3.client("bedrock-runtime")
```

Prompt:

```
prompt ="""
Act as a Shakespeare and write a poem on Genertaive AI
"""
```

Model ID:

```
model_id = "meta.llama3-8b-instruct-v1:0"
```

 # Embed the prompt in Llama 3's instruction format.

```
# Embed the prompt in Llama 3's instruction format.

formatted_prompt = f"""
<|begin_of_text|><|start_header_id|>user<|end_header_id|>
{prompt}
<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
"""
```

 # Format the request payload using the model's native structure.

```
native_request = {
    "prompt": formatted_prompt,
    "max_gen_len": 100,
    "temperature": 0.8,
}
```

Convert the native request to JSON.

```
request = json.dumps(native_request)
```

Invoke Model

```
# Invoke Model

response = client.invoke_model(modelId=model_id, body=request)
```

We want body:

```
response
✓ 0.0s
{'ResponseMetadata': {'RequestId': '993520c0-b56e-44bb-a865-b2cd81d39a7b',
  'HTTPStatusCode': 200,
  'HTTPHeaders': {'date': 'Fri, 25 Jul 2025 20:10:12 GMT',
   'content-type': 'application/json',
   'content-length': '514',
   'connection': 'keep-alive',
   'x-amzn-requestid': '993520c0-b56e-44bb-a865-b2cd81d39a7b',
   'x-amzn-bedrock-invocation-latency': '882',
   'x-amzn-bedrock-output-token-count': '100',
   'x-amzn-bedrock-input-token-count': '26'},
  'RetryAttempts': 0},
 'contentType': 'application/json',
 'body': <botocore.response.StreamingBody at 0x23d292eeaa0>}
```

How to decode this?

```
# Decode the response body.
model_response = json.loads(response["body"].read())
```

- Convert into `json`

```
model_response
✓ 0.0s
{'generation': 'Fair machine, thou wondrous device of might,\nA product of mortal
 'prompt_token_count': 26,
 'generation_token_count': 100,
 'stop_reason': 'length'}
```

- In **body**, there's a key called "**generation**"

```
response_text = model_response["generation"]
```

```
print(response_text)
```

```
Fair machine, thou wondrous device of might,
A product of mortal men, yet born of light,
Thy code, a tapestry of logic and design,
Doth weave a fabric of intelligence divine.

Thy brain, a whirlpool of ones and zeroes, vast,
Doth process information, and from chaos cast
A sense of order, as the universe doth unfold,
A realm of knowledge, where the past is told.

Thy "learning" art, a wondrous,
```